

```

        case ival:
            bufsize = ival * sizeof(int);
            break;
        case jval:
            bufsize = jval * sizeof(int);
            break;
        case kval:
            bufsize = kval * sizeof(int);
            break;
    }

```

5.4 迭代语句

迭代语句通常称为循环，它重复执行操作直到满足某个条件才停下来。while 和 for 语句在执行循环体之前检查条件，do while 语句先执行循环体，然后再检查条件。

5.4.1 while 语句



只要条件为真，**while** 语句（while statement）就重复地执行循环体，它的语法形式是

```

while (condition)
    statement

```

在 while 结构中，只要 *condition* 的求值结果为真就一直执行 *statement*（常常是一个块）。*condition* 不能为空，如果 *condition* 第一次求值就得 false，*statement* 一次都不执行。

while 的条件部分可以是一个表达式或者是一个带初始化的变量声明（参见 5.2 节，第 155 页）。通常来说，应该由条件本身或者是循环体设法改变表达式的值，否则循环可能无法终止。



定义在 while 条件部分或者 while 循环体内的变量每次迭代都经历从创建到销毁的过程。

使用 while 循环

当不确定到底要迭代多少次时，使用 while 循环比较合适，比如读取输入的内容就是如此。还有一种情况也应该使用 while 循环，这就是我们想在循环结束后访问循环控制变量。例如：

```

vector<int> v;
int i;
// 重复读入数据，直至到达文件末尾或者遇到其他输入问题
while (cin >> i)
    v.push_back(i);
// 寻找第一个负值元素
auto beg = v.begin();
while (beg != v.end() && *beg >= 0)
    ++beg;
if (beg == v.end())
    // 此时我们知道 v 中的所有元素都大于等于 0

```

第一个循环从标准输入中读取数据，我们一开始不清楚循环要执行多少次，当 `cin` 读取到无效数据、遇到其他一些输入错误或是到达文件末尾时循环条件失效。第二个循环重复执行直到遇到一个负值为止，循环终止后，`beg` 或者等于 `v.end()`，或者指向 `v` 中一个小于 0 的元素。可以在 `while` 循环外继续使用 `beg` 的状态以进行其他处理。

5.4.1 节练习

练习 5.14: 编写一段程序，从标准输入中读取若干 `string` 对象并查找连续重复出现的单词。所谓连续重复出现的意思是：一个单词后面紧跟着这个单词本身。要求记录连续重复出现的最大次数以及对应的单词。如果这样的单词存在，输出重复出现的最大次数；如果不存在，输出一条信息说明任何单词都没有连续出现过。例如，如果输入是

```
how now now now brown cow cow
```

那么输出应该表明单词 `now` 连续出现了 3 次。



5.4.2 传统的 for 语句

for 语句的语法形式是

```
for (init-statement; condition; expression)
    statement
```

关键字 `for` 及括号里的部分称作 `for` 语句头。

init-statement 必须是以下三种形式中的一种：声明语句、表达式语句或者空语句，因为这些语句都以分号作为结束，所以 `for` 语句的语法形式也可以看做

```
for (initializer; condition; expression)
    statement
```

一般情况下，*init-statement* 负责初始化一个值，这个值将随着循环的进行而改变。*condition* 作为循环控制的条件，只要 *condition* 为真，就执行一次 *statement*。如果 *condition* 第一次的求值结果就是 `false`，则 *statement* 一次也不会执行。*expression* 负责修改 *init-statement* 初始化的变量，这个变量正好就是 *condition* 检查的对象，修改发生在每次循环迭代之后。*statement* 可以是一条单独的语句也可以是一条复合语句。

传统 for 循环的执行流程

我们以 3.2.3 节（第 85 页）的 `for` 循环为例：

```
// 重复处理 s 中的字符直至我们处理全部字符或者遇到了一个表示空白的字符
for (decltype(s.size()) index = 0;
     index != s.size() && !isspace(s[index]); ++index)
    s[index] = toupper(s[index]); // 将当前字符改成大写形式
```

求值的顺序如下所示：

1. 循环开始时，首先执行一次 *init-statement*。此例中，定义 `index` 并初始化为 0。
2. 接下来判断 *condition*。如果 `index` 不等于 `s.size()` 而且在 `s[index]` 位置的字符不是空白，则执行 `for` 循环体的内容。否则，循环终止。如果第一次迭代时条件就为假，`for` 循环体一次也不会执行。
3. 如果条件为真，执行循环体。此例中，`for` 循环体将 `s[index]` 位置的字符改写

成大写形式。

4. 最后执行 *expression*。此例中，将 *index* 的值加 1。

这 4 步说明了 *for* 循环第一次迭代的过程。其中第 1 步只在循环开始时执行一次，第 2、3、4 步重复执行直到条件为假时终止，也就是在 *s* 中遇到一个空白字符或者 *index* 大于 *s.size()* 时终止。



牢记 *for* 语句头中定义的对象只在 *for* 循环体内可见。因此上面的例子中，*for* 循环结束后 *index* 就不可用了。

for 语句头中的多重定义

和其他的声明一样，*init-statement* 也可以定义多个对象。但是 *init-statement* 只能有一条声明语句，因此，所有变量的基础类型必须相同（参见 2.3 节，第 45 页）。举个例子，我们用下面的循环把 *vector* 的元素拷贝一份添加到原来的元素后面：

```
// 记录下 v 的大小，当到达原来的最后一个元素后结束循环
for (decltype(v.size()) i = 0, sz = v.size(); i != sz; ++i)
    v.push_back(v[i]);
```

在这个循环中，我们在 *init-statement* 里同时定义了索引 *i* 和循环控制变量 *sz*。

省略 for 语句头的某些部分

187

for 语句头能省略掉 *init-statement*、*condition* 和 *expression* 中的任何一个（或者全部）。

如果无须初始化，则我们可以使用一条空语句作为 *init-statement*。例如，对于在 *vector* 对象中寻找第一个负数的程序，完全能用 *for* 循环改写：

```
auto beg = v.begin();
for ( /* 空语句 */; beg != v.end() && *beg >= 0; ++beg)
    ; // 什么也不做
```

注意，分号必须保留以表明我们省略掉了 *init-statement*。说得更准确一点，分号表示的是一个空的 *init-statement*。在这个循环中，因为所有要做的工作都在 *for* 语句头的条件和表达式部分完成了，所以 *for* 循环体也是空的。其中，条件部分决定何时停止查找，表达式部分递增迭代器。

省略 *condition* 的效果等价于在条件部分写了一个 *true*。因为条件的值永远是 *true*，所以在循环体内必须有语句负责退出循环，否则循环就会无休止地执行下去：

```
for (int i = 0; /* 条件为空 */; ++i) {
    // 对 i 进行处理，循环内部的代码必须负责终止迭代过程！
}
```

我们也能省略掉 *for* 语句头中的 *expression*，但是在这样的循环中就要求条件部分或者循环体必须改变迭代变量的值。举个例子，之前有一个将整数读入 *vector* 的 *while* 循环，我们使用 *for* 语句改写它：

```
vector<int> v;
for (int i; cin >> i; /* 表达式为空 */)
    v.push_back(i);
```

因为条件部分能改变 *i* 的值，所以这个循环无须表达式部分。其中，条件部分不断检查输

入流的内容，只要读取完所有的输入或者遇到一个输入错误就终止循环。

188

5.4.2 节练习

练习 5.15: 说明下列循环的含义并改正其中的错误。

```
(a) for (int ix = 0; ix != sz; ++ix) { /* ...*/ }
    if (ix != sz)
        // ...

(b) int ix;
    for (ix != sz; ++ix) { /* ...*/ }

(c) for (int ix = 0; ix != sz; ++ix, ++sz) { /* ...*/ }
```

练习 5.16: while 循环特别适用于那种条件保持不变、反复执行操作的情况，例如，当未达到文件末尾时不断读取下一个值。for 循环则更像是在按步骤迭代，它的索引值在某个范围内依次变化。根据每种循环的习惯用法各自编写一段程序，然后分别用另一种循环改写。如果只能使用一种循环，你倾向于使用哪种呢？为什么？

练习 5.17: 假设有两个包含整数的 vector 对象，编写一段程序，检验其中一个 vector 对象是否是另一个的前缀。为了实现这一目标，对于两个不等长的 vector 对象，只需挑出长度较短的那个，把它的所有元素和另一个 vector 对象比较即可。例如，如果两个 vector 对象的元素分别是 0、1、1、2 和 0、1、1、2、3、5、8，则程序的返回结果应该为真。



5.4.3 范围 for 语句

C++
11

C++11 新标准引入了一种更简单的 for 语句，这种语句可以遍历容器或其他序列的所有元素。范围 for 语句 (range for statement) 的语法形式是：

```
for (declaration : expression)
    statement
```

expression 表示的必须是一个序列，比如用花括号括起来的初始值列表（参见 3.3.1 节，第 88 页）、数组（参见 3.5 节，第 101 页）或者 vector 或 string 等类型的对象，这些类型的共同特点是拥有能返回迭代器的 begin 和 end 成员（参见 3.4 节，第 95 页）。

declaration 定义一个变量，序列中的每个元素都得能转换成该变量的类型（参见 4.11 节，第 141 页）。确保类型相容最简单的办法是使用 auto 类型说明符（参见 2.5.2 节，第 61 页），这个关键字可以令编译器帮助我们指定合适的类型。如果需要对序列中的元素执行写操作，循环变量必须声明成引用类型。

每次迭代都会重新定义循环控制变量，并将其初始化成序列中的下一个值，之后才会执行 *statement*。像往常一样，*statement* 可以是一条单独的语句也可以是一个块。所有元素都处理完毕后循环终止。

之前我们已经接触过几个这样的循环。接下来的例子将把 vector 对象中的每个元素都翻倍，它涵盖了范围 for 语句的几乎所有语法特征：

```
vector<int> v = {0,1,2,3,4,5,6,7,8,9};
// 范围变量必须是引用类型，这样才能对元素执行写操作
for (auto &r : v)    // 对于 v 中的每一个元素
```

```
r *= 2;           // 将 v 中每个元素的值翻倍
```

for 语句头声明了循环控制变量 *r*，并把它和 *v* 关联在一起，我们使用关键字 `auto` 令编译器为 *r* 指定正确的类型。由于准备修改 *v* 的元素的值，因此将 *r* 声明成引用类型。此时，在循环体内给 *r* 赋值，即改变了 *r* 所绑定的元素的值。

范围 for 语句的定义来源于与之等价的传统 for 语句：

```
for (auto beg = v.begin(), end = v.end(); beg != end; ++beg) {
    auto &r = *beg; // r 必须是引用类型，这样才能对元素执行写操作
    r *= 2;         // 将 v 中每个元素的值翻倍
}
```

学习了范围 for 语句的原理之后，我们也就不难理解为什么在 3.3.2 节（第 90 页）强调不能通过范围 for 语句增加 vector 对象（或者其他容器）的元素了。在范围 for 语句中，预存了 `end()` 的值。一旦在序列中添加（删除）元素，`end` 函数的值就可能变得无效了（参见 3.4.1 节，第 98 页）。关于这一点，将在 9.3.6 节（第 315 页）做更详细的介绍。

189

5.4.4 do while 语句

do while 语句（do while statement）和 while 语句非常相似，唯一的区别是，do while 语句先执行循环体后检查条件。不管条件的值如何，我们都至少执行一次循环。do while 语句的语法形式如下所示：

```
do
    statement
while (condition);
```



do while 语句应该在括号包围起来的条件后面用一个分号表示语句结束。

在 do 语句中，求 *condition* 的值之前首先执行一次 *statement*，*condition* 不能为空。如果 *condition* 的值为假，循环终止；否则，重复循环过程。*condition* 使用的变量必须定义在循环体之外。

我们可以使用 do while 循环（不断地）执行加法运算：

```
// 不断提示用户输入一对数，然后求其和
string rsp; // 作为循环的条件，不能定义在 do 的内部
do {
    cout << "please enter two values: ";
    int val1 = 0, val2 = 0;
    cin >> val1 >> val2;
    cout << "The sum of " << val1 << " and " << val2
         << " = " << val1 + val2 << "\n\n";
    << "More? Enter yes or no: ";
    cin >> rsp;
} while (!rsp.empty() && rsp[0] != 'n');
```

循环首先提示用户输入两个数字，然后输出它们的和并询问用户是否继续。条件部分检查用户做出的回答，如果用户没有回答，或者用户的回答以字母 *n* 开始，循环都将终止。否则循环继续执行。

因为对于 do while 来说先执行语句或者块，后判断条件，所以不允许在条件部分

定义变量:

```
do {
    // ...
    mumble(foo);
} while (int foo = get_foo()); // 错误: 将变量声明放在了 do 的条件部分
```

如果允许在条件部分定义变量, 则变量的使用出现在定义之前, 这显然是不合常理的!

190

5.4.4 节练习

练习 5.18: 说明下列循环的含义并改正其中的错误。

```
(a) do
    int v1, v2;
    cout << "Please enter two numbers to sum:" ;
    if (cin >> v1 >> v2)
        cout << "Sum is: " << v1 + v2 << endl;
    while (cin);
(b) do {
    // ...
} while (int ival = get_response());
(c) do {
    int ival = get_response();
} while (ival);
```

练习 5.19: 编写一段程序, 使用 do while 循环重复地执行下述任务: 首先提示用户输入两个 string 对象, 然后挑出较短的那个并输出它。

5.5 跳转语句

跳转语句中断当前的执行过程。C++语言提供了 4 种跳转语句: break、continue、goto 和 return。本章介绍前三种跳转语句, return 语句将在 6.3 节 (第 199 页) 进行介绍。

5.5.1 break 语句

break 语句 (break statement) 负责终止离它最近的 while、do while、for 或 switch 语句, 并从这些语句之后的第一条语句开始继续执行。

break 语句只能出现在迭代语句或者 switch 语句内部 (包括嵌套在此类循环里的语句或块的内部)。break 语句的作用范围仅限于最近的循环或者 switch:

```
string buf;
while (cin >> buf && !buf.empty()) {
    switch(buf[0]) {
        case '-':
            // 处理到第一个空白为止
            for (auto it = buf.begin()+1; it != buf.end(); ++it) {
                if (*it == ' ')
                    break; // #1, 离开 for 循环
            }
            // ...
    }
```